

Dense Expands, Sparse Anchors: Channel-Asymmetric LLM Query Expansion for Hybrid Retrieval

Chunran Zhang

School of Computing and Artificial Intelligence
Southwest Jiaotong University
Chengdu, China
chronis@my.swjtu.edu.cn

Abstract

LLM-based query expansion augments queries with generated document-like evidence. In hybrid retrieval, however, most evaluations fuse fixed top- L dense and sparse rankings. The cutoff determines both which cross-channel contributions enter fusion and how much of each ranking is accessed, so expansion gains measured at one cutoff may not persist at another. We address this problem by evaluating complete-list fusion quality together with the dense and sparse depths at which its ordered top- K becomes fixed.

We further introduce channel-asymmetric query expansion. A language model generates complementary reference passages that preserve the original query intent. Orthogonal residual expansion adds their new semantic directions to the dense query representation, while score-product anchoring uses their lexical evidence to rerank documents matched by the original sparse query. We evaluate the method on seven retrieval benchmarks through top- L sensitivity analysis, complete-list fusion, access-depth replay, controlled comparisons, and robustness experiments.

1 Introduction

LLM-based query expansion has become a common approach to improving retrieval effectiveness (Gao et al., 2023; Wang et al., 2023; Zhang et al., 2024). For hybrid retrieval, its gains are typically measured at a preset top- L cutoff using metrics such as nDCG and recall (Zhang et al., 2024; Liu and Zhang, 2025). Recent work on fixed top- L fusion shows that L does more than control access depth: changing the cutoff can also change which candidates and cross-channel contributions enter the fused ranking (Zhang, 2026). Consequently, a gain observed at one L reflects both the query expansion and the chosen cutoff; at another L , its magnitude—and even the ordering between methods—may change. Evidence from a single

cutoff therefore does not isolate the effect of query expansion.

To remove this confound, we evaluate the retrieval effectiveness of each query construction using complete-list fusion, without truncating either channel (Zhang, 2026). We then reveal the dense and sparse rankings progressively and record the per-channel depths at which the same ordered top- K becomes fixed. This separates two questions: what fused ranking does an expansion produce, and how much ranked evidence is required to determine it? The answers need not move together. An expansion may improve retrieval effectiveness while pushing decisive evidence deeper into one channel. We therefore ask whether query expansion can improve effectiveness without increasing the access depth of either channel.

Meeting both goals requires treating dense and sparse retrieval differently. Prior studies show that expansion gains vary across retrievers, and that dense and sparse retrieval can favor different forms of generated evidence (Weller et al., 2024; Li et al., 2026). Our NFCorpus analysis shows what happens when one shared expansion is applied to both channels. When the same generated passages are used in both channels, retrieval effectiveness improves and fewer dense ranks are needed to determine the fused result, yet more sparse ranks are required. The unexpanded sparse query matches only a small portion of the corpus; the added terms make far more documents eligible for sparse ranking, so the list no longer exhausts early. The same generated content therefore supplies useful semantic evidence to the dense retriever while spreading lexical evidence across a much larger sparse list. A shared expansion cannot control these two effects separately.

We address this asymmetry with channel-asymmetric query expansion. The LLM generates several complementary reference passages around the original query. For dense retrieval, orthogo-

nal residual expansion preserves the original query direction and adds only the new semantic component supplied by the references. For sparse retrieval, score-product anchoring multiplies each document’s scores under the unexpanded and expanded queries. Documents matched only by generated terms receive no anchored score, while the new terms can still reorder documents supported by the original query. Together, these operations broaden semantic coverage without broadening sparse support: Dense expands; sparse anchors.

The evaluation mirrors this argument. A top- L sensitivity study first examines whether the measured gains and relative ranking of query expansion methods change with the cutoff. We then evaluate complete-list fusion on seven public retrieval benchmarks, comparing our method with the unexpanded query, shared expansion, and representative generative expansion methods in terms of retrieval effectiveness and per-channel access depth. Controlled experiments examine the contributions of complementary reference generation, orthogonal residual expansion, and score-product anchoring, followed by robustness analyses across generation and retrieval settings. Together, these experiments test whether channel-asymmetric query expansion can produce better fused rankings while reducing the access required from both channels.

2 Related Work

2.1 LLM-Based Query Expansion

Short queries often omit expressions that appear in relevant documents. LLM-based query expansion supplies such expressions by generating text that resembles a relevant document. HyDE generates a hypothetical document and encodes it for dense retrieval (Gao et al., 2023), whereas Query2doc appends a generated pseudo-document to the original query for sparse and dense retrieval (Wang et al., 2023). Although their retrieval procedures differ, both methods use generated document-side language to enrich the information available from the original query.

Later work broadens the evidence produced by the LLM. Jagerman et al. elicit passages, keywords, and reasoning traces through different prompts (Jagerman et al., 2023), while MuGI samples multiple pseudo-references to cover alternative relevant expressions (Zhang et al., 2024). These approaches increase the variety of information available for retrieval, but the added content is not uniformly

useful. Its effect varies across queries, datasets, and retrievers, and irrelevant generations may reduce retrieval effectiveness (Weller et al., 2024).

2.2 Integrating Expansions into Dense and Sparse Retrieval

Generated evidence can enter retrieval at different stages. Query2doc incorporates it into the query text (Wang et al., 2023), whereas MuGI additionally pools contextualized dense representations derived from multiple references (Zhang et al., 2024). Word2Passage reweights generated terms according to their reference granularity (Choi et al., 2025); Exp4Fuse combines rankings obtained from the original and expanded queries (Liu and Zhang, 2025); and DVCQR constructs separate lexical and semantic views for sparse and dense retrieval (Li et al., 2026). Existing methods therefore integrate generated evidence at the textual, representation, term-weighting, or ranking level.

Our method jointly designs reference generation and channel integration. It generates complementary passages that preserve the original query intent, then uses the same evidence asymmetrically: dense retains its new semantic directions, whereas sparse uses its terms only to refine matches supported by the original query. Unlike multi-reference pooling or independently generated retrieval views, our construction couples intent-preserving generation with dense expansion and sparse anchoring.

2.3 Hybrid Fusion under Fixed Cutoffs

Query-expansion methods commonly evaluate retrieval effectiveness after taking a preset number of results from each ranking. MuGI follows this protocol for sparse, dense, and hybrid retrieval (Zhang et al., 2024), while Exp4Fuse combines fixed-depth rankings produced by the original and expanded queries (Liu and Zhang, 2025). In hybrid fusion, however, the cutoff determines both how much of each ranking is available and which cross-channel contributions enter the fused result. Reported expansion gains may therefore change with the selected depth.

EAHR separates the fused result from the depth needed to determine it by defining the target over complete rankings and progressively revealing each channel (Zhang, 2026). We use this distinction to evaluate query expansion. Each query construction is first assessed by the effectiveness of its complete-list fused ranking; dense and sparse access depths are then recorded when replay fixes the same or-

dered top- K . This evaluates both the ranking produced by an expansion and the concentration of evidence within its two channels.

3 Method

3.1 Method Overview

Given an original query q_o , the generator produces a set of complementary reference passages $R = \{r_1, \dots, r_n\}$ that preserve its intent while adding possible document-side expressions. The same references are then used to construct two channel-specific rankings. Orthogonal residual expansion adds their new semantic directions to the dense query representation. Score-product anchoring combines the original and expanded sparse scores for each document, allowing generated terms to reorder documents matched by the original query without expanding its lexical support. The resulting dense and sparse rankings are fused to produce the final result. Throughout this process, the original query provides the base and the generated passages provide incremental evidence: Dense expands; Sparse anchors.

3.2 Complementary Evidence Generation

Given q_o , the generator G produces n complementary reference passages:

$$R = G(q_o) = \{r_1, \dots, r_n\}.$$

Each passage is written as a plausible excerpt from a relevant document and introduces a different alias, technical term, or paraphrase. The prompt preserves the entities, relations, numbers, polarity, temporal conditions, scope, and constraints of q_o , while prohibiting broader information needs or invented answers. The passages therefore vary in expression while describing the same underlying query intent.

3.3 Channel-Asymmetric Query Construction

3.3.1 Orthogonal Residual Expansion

Let f_D denote the dense encoder. We first encode the original query as a unit vector

$$\mathbf{v}_o = \text{norm}(f_D(q_o)),$$

and average the representations obtained by concatenating q_o with each reference passage:

$$\bar{\mathbf{v}}_r = \frac{1}{n} \sum_{i=1}^n f_D(\text{concat}(q_o, r_i)).$$

We remove from $\bar{\mathbf{v}}_r$ the component parallel to $\mathbf{v}_o$, then add the remaining orthogonal residual to the original query:

$$\mathbf{e}_D = \text{norm}\left(\mathbf{v}_o + \bar{\mathbf{v}}_r - (\mathbf{v}_o^\top \bar{\mathbf{v}}_r) \mathbf{v}_o\right).$$

The original query remains the base direction, while the reference passages contribute only semantic directions not already represented by it. If the references provide no orthogonal component, then $\mathbf{e}_D = \mathbf{v}_o$. Documents are ranked by their similarity to $\mathbf{e}_D$, producing the dense ranking π_D .

3.3.2 Score-Product Anchoring

For sparse retrieval, we form an expanded query

$$q_r = \text{concat}(q_o, r_1, \dots, r_n).$$

A non-negative sparse scorer s_S evaluates each document d under the original and expanded queries:

$$s_o(d) = s_S(q_o, d), \quad s_r(d) = s_S(q_r, d).$$

We define the anchored score as their document-wise product:

$$s_A(d) = s_o(d) s_r(d).$$

A document receives a nonzero anchored score only if it matches both queries. Because q_r contains q_o , the anchored ranking retains the lexical support of the original query, while the generated terms change the relative scores within that support. Sorting documents by $s_A(d)$ produces the sparse ranking π_S .

3.4 Quality and Access Analysis

After constructing the dense ranking π_D and sparse ranking π_S , we fuse their complete rankings using weighted reciprocal rank fusion:

$$F(d) = \frac{w_D}{\kappa + \text{rank}_{\pi_D}(d)} + \frac{w_S}{\kappa + \text{rank}_{\pi_S}(d)}.$$

A missing rank contributes zero. Documents are ordered by $F(d)$, with document identifiers used to break ties, and the first K documents form the ordered result T_K . Retrieval effectiveness is evaluated on T_K .

We then replay incremental access to π_D and π_S . After reading L_c items from channel c , the largest contribution of its next unread item is

$$u_c(L_c) = \frac{w_c}{\kappa + L_c + 1}.$$

These bounds determine whether unread ranks can still change the members or order of the current top- K . If they can, the replay reads from the channel with the larger u_c , breaking ties in favor of Dense; otherwise, it stops and records (L_D, L_S) . The resulting measurements describe both the retrieval quality produced by each query construction and the depth at which its fused result becomes fixed.

4 Experiments

We first test whether fixed Top- L changes the conclusions drawn about query expansion. We then evaluate the retrieval quality and channel-specific access depth produced by each query construction. Further experiments separate the effects of complementary evidence generation, orthogonal residual expansion, and score-product anchoring.

4.1 Evaluation Protocol

To measure sensitivity to the cutoff, we vary

$$L \in \{10, 20, 50, 100, 200, 500, 1000\}.$$

At each L , the top L results from the dense and sparse rankings are fused using equally weighted WRRF. We compute nDCG@10 and Recall@20, then compare both the gain over the unexpanded query and the ordering of expansion methods across cutoff depths.

The remaining experiments fuse the complete dense and sparse rankings produced by each query construction. For method m , the fused score of document d is

$$S_m(d) = \frac{1}{60 + r_D^m(d)} + \frac{1}{60 + r_S^m(d)},$$

where $r_D^m(d)$ and $r_S^m(d)$ are its one-based ranks in the two channels. The ordered top-20 obtained from this fusion is used to compute nDCG@10 and Recall@20.

Access depth is measured using the replay procedure in Section 3.4. We record the stopping depths (L_D, L_S) , their changes relative to the unexpanded query, and the proportion of queries for which both depths decrease. We also report the size and exhaustion rate of the sparse support.

4.2 Datasets and Retrieval Setup

We evaluate on seven English retrieval benchmarks: SciFact, NFCorpus, TREC-COVID, FiQA, ArguAna, Touché-2020, and SCIDOCS. They cover scientific claim verification, biomedical and financial retrieval, argument retrieval, and scientific document recommendation.

For the scale analysis, we construct nested TREC-COVID corpora containing 25,000, 50,000, 100,000, and all documents. Each snapshot retains the judged relevant documents, while the remaining documents are added in a fixed hash order. The smaller corpus is therefore a strict subset of every larger one.

The primary generator is Qwen2.5-7B-Instruct. For each query, we run three independent generations and produce five reference passages per run. We repeat the analysis with Mistral-7B-Instruct-v0.3 on 100 queries selected by a fixed hash of query identifiers from each of FiQA, ArguAna, Touché-2020, and SCIDOCS.

Dense retrieval uses BGE-small-en-v1.5 with normalized query and document vectors ranked by dot product. Sparse retrieval uses Lucene BM25 through Pyserini 2.3.0, with $k_1 = 0.9$ and $b = 0.4$. Contriever provides a second dense encoder for the robustness analysis. Score ties are broken by document identifier.

4.3 Comparison Methods

For the unexpanded-query baseline, both Dense and Sparse use q_o . Shared expansion keeps the reference passages generated by our method but replaces its two channel-specific operators. Dense uses the mean representation of $[q_o; r_i]$ across the five passages, and Sparse retrieves with $q_r = \text{concat}(q_o, r_1, \dots, r_5)$. Shared expansion and our method therefore use identical generated evidence; they differ only in how that evidence is converted into Dense and Sparse rankings.

Our method combines complementary evidence generation with orthogonal residual expansion for Dense and score-product anchoring for Sparse.

We also compare with three representative generative expansion methods. HyDE encodes a generated hypothetical document for dense retrieval (Gao et al., 2023). Query2doc appends a generated pseudo-document to the query (Wang et al., 2023). MuGI samples multiple pseudo-references, pools their contextualized dense representations, and uses query-weighted concatenation for sparse

retrieval (Zhang et al., 2024).

4.4 Mechanism and Robustness Analysis

We first separate the contributions of the two channel-specific operators. Using the same complementary reference passages, we evaluate four constructions: the unexpanded query in both channels, orthogonal residual expansion only, score-product anchoring only, and the combination of both operators.

We then examine the generation strategy while keeping the two operators fixed. Reference passages are produced using either our complementary-evidence prompt or the MuGI multi-reference prompt, with the same generator, number of passages, and generation runs.

Further experiments vary the number of reference passages over $\{1, 3, 5\}$. For Sparse, score-product anchoring is compared with an original-query binary mask and retrieval using only the generated passages. We also vary the WRRF constant over $\{2, 20, 60, 100\}$, replace Qwen2.5-7B-Instruct with Mistral-7B-Instruct-v0.3, and replace BGE-small-en-v1.5 with Contriever. The nested TREC-COVID corpora measure how retrieval quality and access depth change with corpus size.

4.5 Statistical Analysis

The three generation runs are averaged within each query. Results are reported for each dataset and as an equal-weight macro average across datasets. We obtain 95% confidence intervals from 10,000 stratified bootstrap samples, resampling queries within each dataset.

The primary comparisons are our method against the unexpanded query and shared expansion. We use 10,000 stratified paired sign-flip randomizations and apply Holm correction across $n\text{DCG}@10$, $\text{Recall}@20$, L_D , and L_S for each comparison. Access measurements refer to logical replay depth rather than wall-clock latency.

References

- Jeonghwan Choi, Minjeong Ban, Minseok Kim, and Hwanjun Song. 2025. [Word2Passage: Word-level importance re-weighting for query expansion](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8276–8296, Vienna, Austria. Association for Computational Linguistics.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. [Precise zero-shot dense retrieval without relevance labels](#). In *Proceedings of the 61st Annual*

Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1762–1777, Toronto, Canada. Association for Computational Linguistics.

- Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky. 2023. [Query expansion by prompting large language models](#). *Preprint*, arXiv:2305.03653.
- Chenyi Li, Xinhui Tu, and Zaixiang Wang. 2026. [DVCQR: Dual-view conversational query rewriting with stage-wise reinforcement learning](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22993–23014, San Diego, California, United States. Association for Computational Linguistics.
- Lingyuan Liu and Mengxiang Zhang. 2025. [Exp4Fuse: A rank fusion framework for enhanced sparse retrieval using large language model-based query expansion](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 163–173, Vienna, Austria. Association for Computational Linguistics.
- Liang Wang, Nan Yang, and Furu Wei. 2023. [Query2doc: Query expansion with large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9414–9423, Singapore. Association for Computational Linguistics.
- Orion Weller, Kyle Lo, David Wadden, Dawn Lawrie, Benjamin Van Durme, Arman Cohan, and Luca Soldaini. 2024. [When do generative query and document expansions fail? a comprehensive study across methods, retrievers, and datasets](#). In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 1987–2003, St. Julian’s, Malta. Association for Computational Linguistics.
- Chunran Zhang. 2026. [Exact adaptive hybrid retrieval without fixed top-1 cutoffs](#). *Preprint*, arXiv:2608.07152.
- Le Zhang, Yihong Wu, Qian Yang, and Jian-Yun Nie. 2024. [Exploring the best practices of query expansion with large language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1872–1883, Miami, Florida, USA. Association for Computational Linguistics.

A Generation and Reproducibility Details

A.1 Complementary Reference Generation

We use the same revision of Qwen2.5-7B-Instruct for the model and tokenizer:

```
fe11104b620d588ccc049ff6631dd3ea002e3d98
```

The unquantized BF16 model runs with MLX-LM 0.31.2. Decoding uses a temperature of 0.7, top- p of 0.9, top- k of 20, and a maximum of 256 new tokens. Queries are sorted by identifier and

processed in batches of eight. Each batch seed is derived from the protocol version, dataset, prompt hash, and batch index using the first 64 bits of their SHA-256 digest.

The system prompt is:

You generate evidence-like reference passages for information retrieval.

Given one search query, return exactly five concise, complementary passages that could plausibly occur in relevant documents. Preserve every named entity, relation, number, polarity, temporal condition, scope, and constraint in the query. Add useful aliases, technical terminology, and paraphrases, but do not broaden the information need or invent a specific answer. Each passage must be one sentence of at most 25 words.

Return one single-line JSON array containing exactly five distinct strings. Count the five array items before responding. Do not output an object key, query identifier, Markdown, placeholders, or any other text. Every item must contain a substantive passage.

The query is supplied in a canonical JSON object with a single query field. XGrammar 0.2.1 constrains the response to a JSON array containing exactly five strings. Validation rejects empty passages, placeholders, and duplicates. An invalid response is retried once with the same seed, the previous response, and validation feedback. If the retry also fails, the failure is recorded and the corresponding query and generation run fall back to the unexpanded query.

The second generator is Mistral-7B-Instruct-v0.3 at revision

c170c708c41dac9275d15a8fff4eca08d52bab71.

It uses the same decoding, batching, and validation settings.

A.2 Query Construction

All averaging, projection, and normalization operations in orthogonal residual expansion use float32. Shared expansion uses the same representations of the query-reference pairs, but averages and normalizes them without removing the component parallel to the original query. Each pair is encoded as query [SEP] reference.

For sparse retrieval, the expanded query contains one copy of the original query followed by the selected reference passages. BM25 scores under the original and expanded queries are stored separately before applying the document-wise score product. The binary-mask ablation retains the expanded-query score for documents matched by both queries. The references-only ablation removes the original

query before retrieval. Our construction neither repeats the original query nor uses the MuGI repetition coefficient.

A.3 Baseline Reproduction

All generative baselines use the same primary generator so that their differences mainly reflect query construction rather than model capacity. The exact prompts are included in the accompanying repository.

HyDE generates eight hypothetical texts per run, encodes them independently as documents, and averages their normalized representations. Its sparse channel uses the original query. FiQA, ArguAna, SciFact, and TREC-COVID use task-specific generation instructions; the remaining datasets use a general retrieval instruction. HyDE allows up to 512 new tokens per generation.

Query2doc generates one pseudo-relevant document of at most 120 words. Dense retrieval encodes the concatenation of the original query and pseudo-document. Sparse retrieval concatenates five copies of the original query with the pseudo-document.

MuGI generates five pseudo-references. Dense retrieval averages the representations obtained by concatenating the original query with each reference. Sparse retrieval repeats the original query

$$\left\lfloor \frac{|R|}{|q_o|\beta} \right\rfloor$$

times before appending all references, where $|\cdot|$ denotes character length and $\beta = 4$. We reproduce MuGI’s generation and query integration components without its pseudo-relevance feedback calibration.

A.4 Retrieval Implementation

BGE-small-en-v1.5 uses revision

5c38ec7c405ec4b44b94cc5a9bb96e735b38267a.

Queries are prefixed with “Represent this sentence for searching relevant passages:” and encoded with a maximum length of 512 tokens. Contriever uses mean pooling and revision

2bd46a25019aeea091fd42d1f0fd4801675cf699.

All query and document embeddings are normalized.

Pyserini 2.3.0 runs with Java 21 and uses DefaultEnglishAnalyzer. A separate Lucene index is built for each dataset and each TREC-COVID snapshot, so BM25 collection statistics are

computed within the corresponding corpus. Documents with equal retrieval or fusion scores are ordered by the UTF-8 order of their identifiers.

A.5 Access-Depth Replay

The replay begins with empty Dense and Sparse prefixes. After l_c items have been read from channel c , the maximum contribution of its next unread document is

$$u_c(l_c) = \frac{1}{60 + l_c + 1}.$$

The bound is zero after a channel is exhausted. At each step, the replay reads from the channel with the larger next-unread bound, breaking ties in favor of Dense.

For an observed document, its accumulated contribution is a score lower bound. If the document has not appeared in the other channel, that channel’s next-unread contribution supplies an additional upper bound. A document unseen in both channels is bounded by the sum of the two next-unread contributions.

The replay stops when these bounds certify both the membership and order of the current top-20. The stopping depths are recorded as $(L_D, L_S) = (l_D, l_S)$.

A.6 Artifact Traceability

Each generation record contains the raw response, parsed passages, prompt and model hashes, decoding parameters, seed, retry history, runtime lock, and code commit. Dense and Sparse rankings are stored by channel in a compressed ranking store and assigned SHA-256 hashes. Each query-level result contains the method, generation run, effectiveness metrics, (L_D, L_S) , sparse support and exhaustion status, ordered top-20, and hashes of the generation, rankings, method configuration, and replay trace.

The Python environment is fixed by `uv.lock`, and the Java version is checked before retrieval. Before held-out evaluation, the code, configurations, prompts, source inputs, model artifacts, generations, and ranking stores are recorded in a hash manifest. All reported tables are reconstructed from the resulting query-level records.